

CAB230 Rentals API - Client Side Application Report

Colby Oosthuizen | n11595388 | Generated 30 April 2026

This report follows the structure of the provided university report template. It explains what the application does, how the required API endpoints are used, how the React application is organised, what was tested, and what limitations remain.

Introduction

Purpose & description

I built CAB230 Rentals as a client-side React single-page application for browsing Australian rental listings from the CAB230 Rentals REST API. The app lets users search rentals by state, property type, and postcode, inspect individual rental details, view a map of the property location, register and log in, submit star ratings, and review properties they have previously rated.

The main design goal was to make the app feel like a practical property-search tool rather than a basic API demo. The rental search page uses AG Grid with the Infinite Row Model, so the table fetches more rows as the user scrolls instead of forcing manual next/previous paging. Sorting is sent back to the API using `sortBy` and `sortOrder`, which means the full dataset is sorted server-side rather than only sorting the rows currently visible in the browser.

The app also includes postcode filtering, a protected Rated Rentals page, automatic logout when the API returns a 401 response, formatted rental descriptions, and map rendering with Leaflet. Bootstrap is used for the general layout so that forms, navigation, buttons, and cards stay consistent across the site.

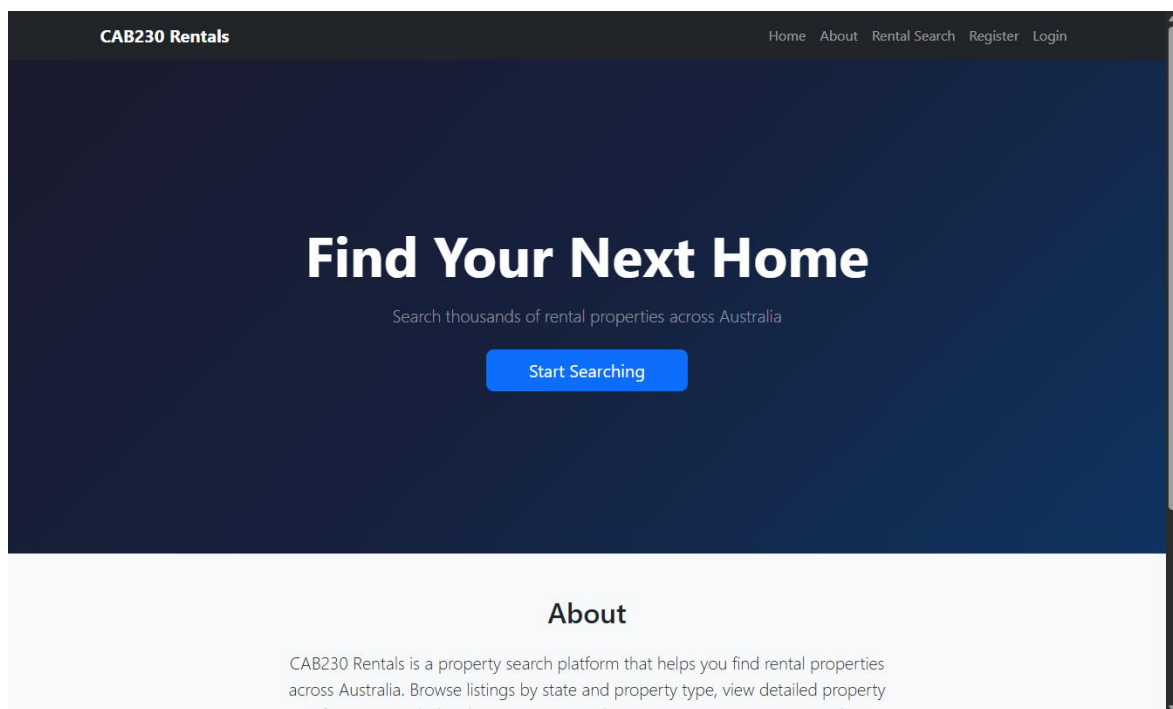


Figure 1. Landing page and main navigation.

CAB230 Rentals

[Home](#) [About](#) [Rental Search](#) [Register](#) [Login](#)

Rental Search

State

Property Type

Postcode

All States

All Types

e.g. 4000

Search

Use the filters above to search for rentals.

Figure 2. Rental search filters before a search is run.

Completeness and Limitations

The core application is complete against the main client-side requirements: users can search listings, filter them, open detailed rental pages, view map locations, authenticate, rate rentals, and view rated rentals. The implementation goes beyond a simple table by using AG Grid infinite scrolling and server-side sorting.

There are still limitations. Some authenticated features require a valid user account and live API availability, so they cannot be meaningfully used offline. The Rated Rentals page enriches ratings by fetching each related rental one by one, which is acceptable for normal use but could be inefficient for users with a very large number of ratings. The same-postcode suggestions show only the first page returned by the API, so they are useful but not exhaustive.

Use of End Points

The app is organised around the API endpoints. Data fetching is centralised through service modules in `src/services`, with shared response handling in `apiUtils.js`.

[/rentals/states](#)

Used when Rental Search loads. It populates the State dropdown with the values returned by the API, including ACT, NSW, NT, Qld, SA, Tas, Vic, and WA. This avoids hardcoding the available states.

[/rentals/property-types](#)

Used when Rental Search loads. It populates the property type dropdown with values such as apartment, house, studio, townhouse, unit, and villa.

[/rentals/search](#)

The main search endpoint. The AG Grid datasource calls it with `page`, `state`, `propertyType`, `postcode`, `sortBy`, and `sortOrder` parameters. The response data array is passed to AG Grid, and `pagination.total` is used so the grid knows the total row count for infinite scrolling.

</rentals/{id}>

Used when a user opens an individual listing and when the Rated Rentals page needs to enrich rating records with full property details. It returns the title, rent, description, coordinates, postcode, agency name, bedrooms, bathrooms, parking spaces, and rating statistics.

</ratings>

Used on the protected Rated Rentals page. It returns the rentals the current authenticated user has rated. The page then fetches each rental detail so the user sees meaningful property information rather than only rental IDs.

</ratings/rentals/{id}>

Used with GET to retrieve the current user rating for a rental and POST to submit or update a rating. The star rating component uses this endpoint once the user is authenticated.

</user/register>

Used by the Register page. It sends email and password to create an account. On success, the user is redirected to Login with a confirmation message.

</user/login>

Used by the Login page. It returns a bearer token, which is stored in localStorage and used by authenticated rating requests. The login flow preserves the page the user was trying to access and returns them there after login.

CAB230 Rentals

HomeAboutRental SearchRegisterLogin

← Back to search

Three Bedroom Town House In Quiet Complex

Unit 4/16 Damala Street, Waramanga, ACT 2611

Weekly Rent

\$560

Property Type

townhouse

Bedrooms

3

Bathrooms

1

Parking

2

Agency

Momentum Property

Description

Positioned in a quiet and friendly complex, this property offers a peaceful living experience. Located within strolling distance to Cooleman Court, Woden Plaza, the Canberra Hospital, and a number of good schools, this home will suit your needs.

Features Include;

* Three bedrooms, all with built-in robes

* Brand new carpets

* Updated Bathroom

* European laundry inside bathroom

* Lounge/Dining room

* Gas cooking and hot water

* Double carport

* Fully enclosed backyard

Location

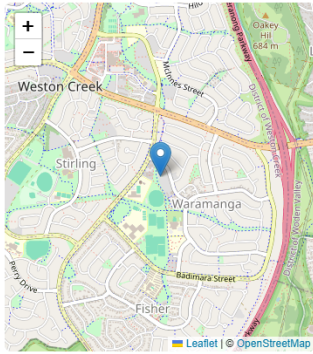


Figure 3. Individual rental endpoint data displayed with details, formatted description, and map.

Login

Log in to rate properties and return to your Rated Rentals dashboard.

Email

Password

Login

Don't have an account? [Sign up here](#)

Figure 4. Login page for authenticated rating features.

Modules Used

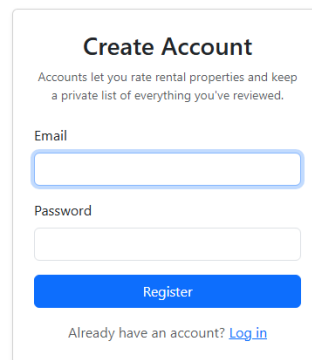
Module	How it is used	Reference
react	Builds the component-based user interface.	https://react.dev
react-dom	Mounts the React application into the browser DOM.	https://react.dev
react-router-dom	Provides BrowserRouter, Routes, Link, navigation, route parameters, and protected-route redirects.	https://reactrouter.com
ag-grid-community	Provides the data grid and Infinite Row Model for rental search results.	https://www.ag-grid.com
ag-grid-react	Provides the React wrapper for AG Grid.	https://www.ag-grid.com/react-data-grid/
leaflet	Displays interactive maps and markers for rental locations.	https://leafletjs.com
react-leaflet	Provides React components for Leaflet maps.	https://react-leaflet.js.org
vite	Runs the development server and builds the static production files.	https://vite.dev
bootstrap	Provides responsive layout, navbar, forms, cards, tables, alerts, and buttons.	https://getbootstrap.com

Application Design

Navigation and Layout

The application uses a persistent top navigation bar with links to Home, About, Rental Search, Register/Login, or Rated Rentals/Log Out depending on authentication state. This keeps the main actions visible without overwhelming the user. The layout uses Bootstrap containers, rows, cards, and tables so the pages feel consistent.

The main flow is Home -> Rental Search -> Individual Rental -> Login/Register if the user wants to rate -> Rated Rentals. This mirrors how a user would naturally browse rentals: first search broadly, then inspect details, then save judgement through a rating only if they care about the property.



Create Account

Accounts let you rate rental properties and keep a private list of everything you've reviewed.

Email

Password

Register

Already have an account? [Log in](#)

Figure 5. Register page explaining why an account is useful.

Usability and Quality of Design

The strongest part of the design is that the search page is simple: three filters and one Search button. The table is not shown until the user searches, which avoids presenting an empty grid before the user has done anything. Infinite scrolling reduces friction for large result sets, and clicking a row to open details follows normal table/listing behaviour.

The individual rental page is also organised around the user task. Key facts such as rent, property type, bedrooms, bathrooms, parking, and agency are shown in compact summary cards. The full description is formatted into readable paragraphs, and the map sits beside the details on larger screens.

The main compromises are around feedback and validation. The postcode field currently accepts any text rather than enforcing four digits. AG Grid has built-in loading behaviour, but a more obvious loading state above the table would improve clarity. The rating success alert also remains visible until the user navigates away; auto-dismissing it would feel more polished.

Accessibility

The application partially meets the W3C Priority 1 checklist. Form controls use visible labels and htmlFor/id pairs on the search, login, and register pages. Table headers are present in the Rated Rentals table, and AG Grid supplies column headers for the search table. The app uses clear language in its own interface labels and avoids flickering or rapidly changing content.

There are still accessibility weaknesses. The Leaflet map is visual and should have a stronger text alternative or ARIA label describing that it shows the property location. The star rating control uses star symbols and colour, although the current rating is also displayed as text once a rating exists. Dynamic AG Grid row loading may not be announced clearly to screen reader users.

For Aboriginal and Torres Strait Islander inclusion, the application currently does not include an Acknowledgement of Traditional Owners even though it presents housing and land-related information across Australia. A respectful acknowledgement on the landing page or footer would be appropriate. The app also assumes English language and stable internet access, which may exclude some users in regional or remote communities. Future improvements should consider clearer inclusive language, low-bandwidth behaviour, and links to culturally appropriate housing support resources.

Technical Description

Architecture

The app is structured as a React SPA. App.jsx defines the route map, AuthProvider supplies global authentication state, page components own page-level data fetching and state, reusable components render tables/maps/navigation, and service modules isolate API calls.

Area	Files	Responsibility
Routing	src/App.jsx, ProtectedRoute.jsx	Defines public/protected routes, 404 route, and redirect-to-login behaviour.
Authentication	context/AuthContext.jsx, context/authContextCore.js	Stores token in localStorage, exposes login/logout/useAuth, and listens for auth-expired events.
API services	services/apiUtils.js, rentalsApi.js, ratingsApi.js, authApi.js	Builds API URLs, handles responses, dispatches 401 logout, and exposes endpoint-specific functions.
Search	RentalSearchPage.jsx, RentalTable.jsx	Collects filters, creates AG Grid datasource, fetches pages as the user scrolls, and sends sort parameters to the API.
Details	IndividualRentalPage.jsx, RentalMap.jsx, StarRating.jsx	Loads one rental, formats description, shows map, same-postcode suggestions, and rating controls.
Rated rentals	RatedRentalsPage.jsx, Paginator.jsx	Loads authenticated rating history and enriches each record with rental details.

The most important implementation detail is the AG Grid datasource. RentalTable receives a filters object from RentalSearchPage. When filters change, the component replaces the grid datasource so AG Grid clears its cache and fetches page 1 again. Inside getRows, AG Grid startRow is converted into an API page number, and the first sort model entry becomes sortBy and sortOrder.

Test plan

Test case	Type	Expected result	Result
Landing page loads	Positive	Navbar, hero text, and search call-to-action render.	Pass
Search page loads	Positive	State, property type, postcode, and Search controls appear.	Pass
Search by state Qld	Positive	Grid requests Qld results from /rentals/search.	Pass
Search by postcode 2611	Positive	API returns postcode 2611 rentals; same-postcode suggestions use the same filter.	Pass
Sort by Rent/wk	Positive	Grid sends sortBy=rent and sortOrder to API and receives sorted rows.	Pass
Open rental detail	Positive	Rental details, formatted description, and map render.	Pass
Unauthenticated rating	Negative	Rating is disabled and user is prompted to log in.	Pass
Access /rated while logged out	Negative	ProtectedRoute redirects to Login and preserves destination.	Pass
Invalid URL	Edge	404 page renders rather than a blank screen.	Pass
Expired token	Edge	401 response dispatches auth:expired and AuthProvider logs out.	Pass
Build	Non-functional	npm.cmd run build completes.	Pass
Lint	Non-functional	npm.cmd run lint completes.	Pass

Difficulties / Exclusions / unresolved & persistent errors /

The largest technical difficulty was moving the search table from manual pagination to AG Grid infinite scrolling. It required mapping AG Grid row requests to the API page model and replacing the datasource whenever filters changed. Another issue was that /rentals/{id} does not return the rental id in the response, so the Rated Rentals page needs to carry rentalId forward when merging rating data with rental details.

The main unresolved limitations are not fatal. Authenticated screenshots and final submission packaging still need to be completed before upload. From a code perspective, the biggest future cleanup would be

improving postcode validation, adding stronger accessibility labels to the map and star rating controls, and avoiding a burst of individual rental-detail requests when a user has many ratings.

Extensions (Optional)

Possible future extensions include saving favourite searches, showing rated properties directly in the search table, adding postcode validation, adding an Acknowledgement of Traditional Owners, improving map accessibility, adding a clearer grid loading overlay, and deploying the static build online if the assignment or users require hosted access.

User guide

Open the application and use the navigation bar to move between pages. Select Rental Search to find listings. Choose any combination of State, Property Type, and Postcode, then press Search. Scroll inside the table to load more results automatically. Click a column header to sort the results.

Click any rental row to open its detail page. The detail page shows address, rent, property type, bedrooms, bathrooms, parking, agency, description, map location, same-postcode suggestions, and the rating widget. To rate a property, register for an account, log in, and select one to five stars. Rated Rentals shows the properties the current account has already rated.

CAB230 Rentals

[Home](#) [About](#) [Rental Search](#) [Register](#) [Login](#)

Login

Log in to rate properties and return to your Rated Rentals dashboard.

Email

Password

Login

Don't have an account? [Sign up here](#)

Figure 6. Protected Rated Rentals route redirects unauthenticated users to Login.

Page not found

That rental page does not exist.

[Go home](#)

Figure 7. 404 route for invalid paths.

References

React Documentation. (2024). React. <https://react.dev>

React Router. (2024). React Router. <https://reactrouter.com>

AG Grid. (2024). AG Grid React Data Grid: Infinite Row Model. <https://www.ag-grid.com/react-data-grid/infinite-scrolling/>

Leaflet. (2024). Leaflet documentation. <https://leafletjs.com>

React Leaflet. (2024). React Leaflet documentation. <https://react-leaflet.js.org>

Bootstrap. (2024). Bootstrap documentation. <https://getbootstrap.com>

Vite. (2024). Vite guide. <https://vite.dev>

World Wide Web Consortium. (1999). Web Content Accessibility Guidelines 1.0 checklist. <https://www.w3.org/TR/WAI-WEBCONTENT/full-checklist>

QUT CiteWrite. (2024). Citing and referencing. <https://www.citewrite.qut.edu.au/cite/>

Appendices as you require them

Appendix A: Source code is organised in the cab230-rentals project folder. Appendix B: Screenshots used in this report are stored in cab230-submission/screenshots. Appendix C: Build and lint checks were run locally after the latest GitHub pull.